

THE USE OF BLOCKCHAIN FOR DIGITAL ADVERTISING AND MARKETING

COURSE : CRYPTOCURRENCY AND BLOCKCHAIN DEVELOPMENT

CONTEXT : CA-1 CASE STUDY ON BLOCKCHAIN APPLICATION (MINI-PROJECT)

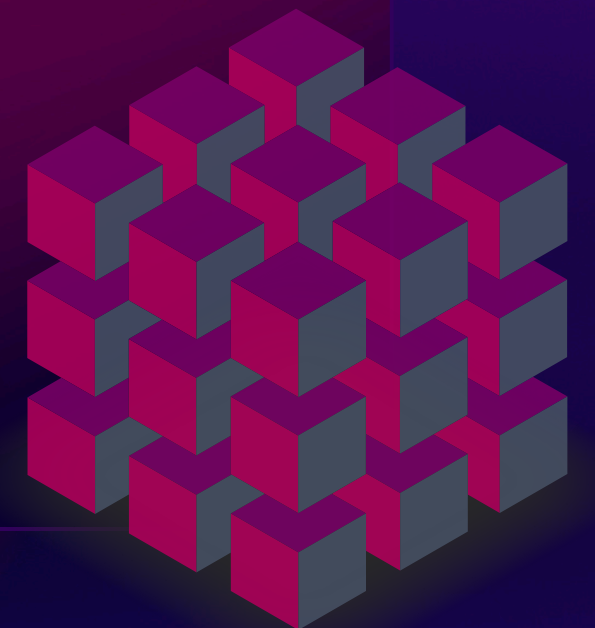
TEAM MEMBERS



SAKSHI KUKREJA 29



NIHAL SINHA 51

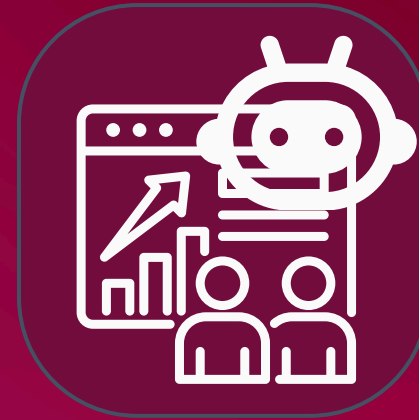


PROBLEMS IN TRADITIONAL DIGITAL ADVERTISING

1. Ad Fraud & Invalid Traffic

- Bots generate fake clicks and impressions
- Click farms artificially inflate engagement
- Domain spoofing misleads advertisers
- Fake conversions reduce campaign ROI

Impact: Billions of dollars lost annually
Result: Advertisers pay for traffic that doesn't exist



2. Lack of Transparency

- Advertisers cannot track exact fund allocation
- Intermediaries operate as “black boxes”
- Hidden markups and unclear bidding processes
- No real-time auditability

Impact: Uncertain ROI and inefficient budget utilization



3. Excessive Intermediaries

Digital ad ecosystem includes:

- Ad networks
- DSPs (Demand-Side Platforms)
- SSPs (Supply-Side Platforms)
- Data brokers
- Each takes a commission.

Impact: Only ~40–60% of advertiser spend reaches publishers
Result: Reduced revenue for content creators



4. Data Privacy & User Exploitation

- Unauthorized tracking through cookies
- User data sold to third parties
- Lack of explicit consent
- Non-compliance risks with regulations (GDPR, etc.)

Impact: Privacy violations and legal liabilities
Result: Loss of user trust



BLOCKCHAIN-BASED SOLUTIONS TO DIGITAL ADVERTISING PROBLEMS

1. Eliminating Ad Fraud with Immutable Tracking

Problem: Fake clicks, bot traffic, domain spoofing

Blockchain Solution:

- Every impression & click is recorded on an immutable ledger
- Cryptographic verification of events
- Fraud detection via transparent audit trails
- Smart contracts release payments only for verified actions

Impact: Reduced invalid traffic & trusted reporting

2. Transparency in Budget Allocation

Problem: Hidden fees & black-box intermediaries

Blockchain Solution:

- Campaign budget locked in smart contract
- All transactions publicly verifiable
- Real-time tracking of fund flow
- No hidden markups

Impact: Advertisers see exactly where money goes

3. Reducing Intermediaries

Problem: DSPs, SSPs, exchanges take large commissions

Blockchain Solution:

- Peer-to-peer marketplace
- Direct advertiser–publisher interaction
- Smart contracts automate bidding and settlement

Impact: Higher revenue for publishers, lower cost for advertisers

BLOCKCHAIN-BASED SOLUTIONS TO DIGITAL ADVERTISING PROBLEMS

4. User Data Ownership & Privacy Protection

Problem: Unauthorized tracking & data misuse

Blockchain Solution:

- Decentralized digital identity
- Users control what data they share
- Permission-based targeting
- Encrypted storage of user consent

Impact: Privacy-first advertising ecosystem

5. Automated & Instant Payments

Problem: Delayed settlements & disputes

Blockchain Solution:

- Smart contracts trigger automatic payouts
- No manual reconciliation
- Conditional payments based on verified engagement

Impact: Faster and dispute-free settlements

6. Decentralized Trust Model

Problem: Centralized dominance by major platforms

Blockchain Solution:

- Distributed ledger removes central authority
- Equal participation rights
- Transparent governance mechanisms

Impact: Fair and competitive ecosystem

DECENTRALIZED ADVERTISING & REWARD MARKETPLACE (DARM)

System Overview

The Decentralized Advertising & Reward Marketplace (DARM) is a blockchain-powered platform that creates a transparent, fraud-resistant, and privacy-preserving digital advertising ecosystem.

It connects advertisers, publishers, and users through smart contracts and token-based incentives eliminating unnecessary intermediaries and ensuring trustless execution.

Core Objective

To build a transparent and automated advertising marketplace where:

1. Campaign budgets are securely locked on blockchain
2. Engagement data is tamper-proof
3. Payments are automatically executed
4. Users are rewarded for genuine attention
5. Privacy is preserved through decentralized identity

WORKING PRINCIPLE

1. Advertiser defines campaign goals and deposits budget into a smart contract

2. Campaign rules are stored immutably on blockchain

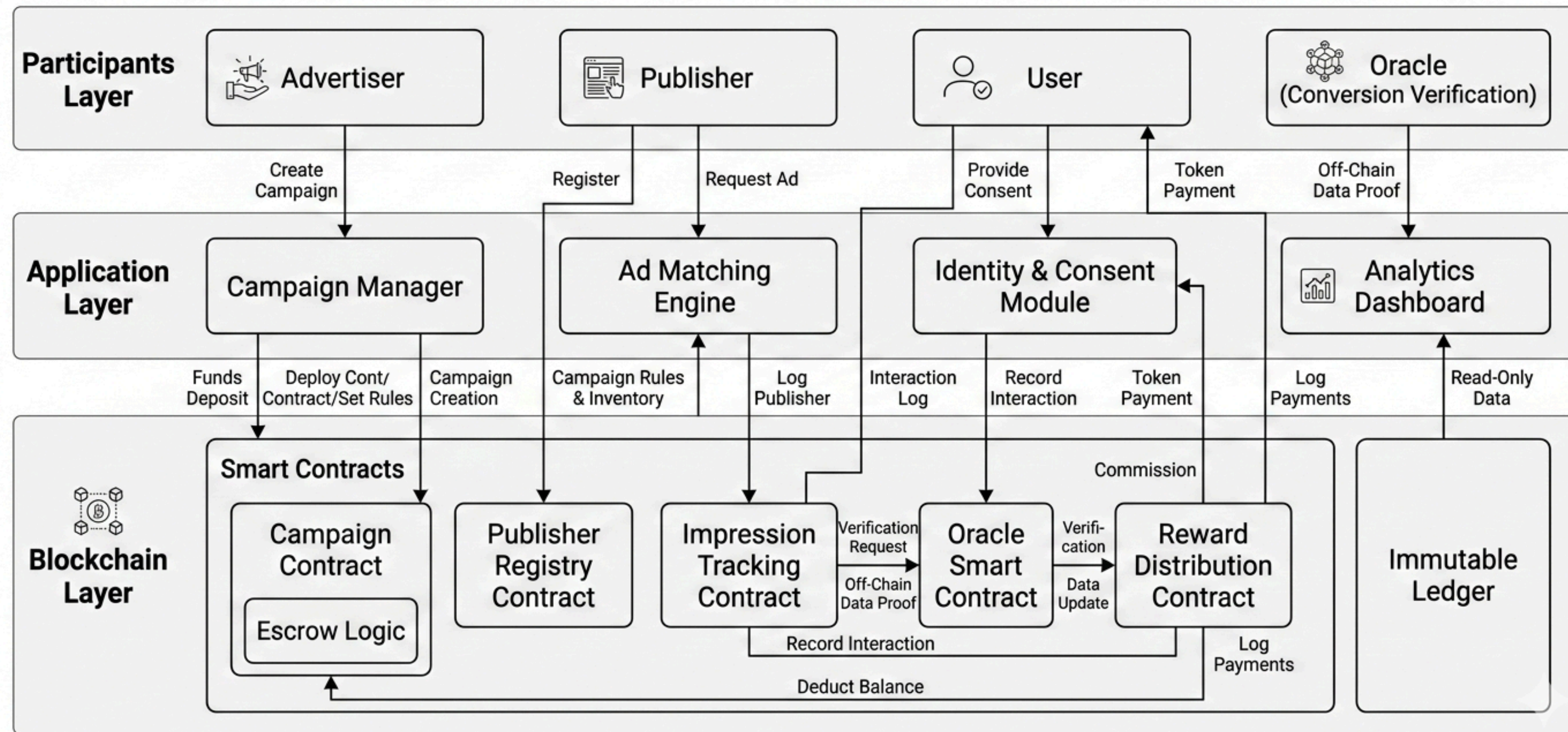
3. Verified user interactions are recorded transparently

4. Smart contracts automatically release rewards based on predefined conditions

5. All transactions remain publicly auditable

BLOCKCHAIN-BASED DIGITAL ADVERTISING ARCHITECTURE

Decentralized Advertising & Reward Marketplace (DARM) – Blockchain Architecture



SMART CONTRACTS IN THE PROPOSED SOLUTION

Campaign Contract

Purpose:

Creates and manages advertising campaigns on the blockchain.

Key Functions:

- Register new campaign
- Store campaign budget
- Define targeting rules
- Activate / pause campaign
- Track remaining fund

Role: Ensures transparent and immutable campaign execution.

Impression Tracking Contract

Purpose:

Records ad impressions and clicks immutably.

Key Functions:

- Log verified impressions
- Log verified clicks
- Store timestamped engagement records
- Prevent duplicate or fraudulent entries

Role: Provides transparent and tamper-proof tracking.

Publisher Registry Contract

Purpose:

Manages publisher onboarding and verification.

Key Functions:

- Register publisher
- Verify publisher identity
- Approve / blacklist publishers
- Maintain publisher reputation score

Role: Ensures only legitimate publishers participate.

Rewards Distribution Contract

Purpose:

Automates token payments to users and publishers.

Key Functions:

- Calculate reward amount
- Transfer tokens to users
- Transfer tokens to publishers
- Refund unused campaign budget

Role: Ensures automatic, transparent, and dispute-free settlements.

IMPLEMENTATION OF DAPP

DEVELOPMENT STACK

1. Blockchain Platform: Ethereum
2. Smart Contract Language: Solidity
3. IDE: Remix IDE
4. Wallet Integration: MetaMask
5. Test Network: Sepolia / Goerli
Testnet
6. Frontend : HTML / JavaScript +
Web3.js

PHASE 1 – EXPERIMENT 7

Creating Smart Contracts & Performing Transactions using Solidity & Remix

Implementation Steps

1. Designed smart contracts for:
 - Campaign creation
 - Budget allocation
 - Interaction logging
 - Reward distribution
2. Written in Solidity and compiled using Remix IDE
3. Deployed contract on Ethereum test network
4. Performed transactions such as:
 - Advertiser depositing campaign funds
 - Logging user interaction
 - Triggering reward payout

PHASE 2 – EXPERIMENT 8

MetaMask Wallet Integration & Live Transactions

Implementation Steps

1. Installed and configured MetaMask wallet
2. Connected MetaMask to Remix
3. Linked wallet address to:
 - Advertiser account
 - Publisher account
 - User account
4. Executed real-time transactions:
 - Funding campaign through wallet
 - Approving smart contract access
 - Receiving reward tokens

IMPLEMENTATION OF DAPP - TRUFFLE SUITE AND GANACHE

STEP 1: Create Truffle Project

```
C:\Users\saksh>truffle init

Starting init...
=====

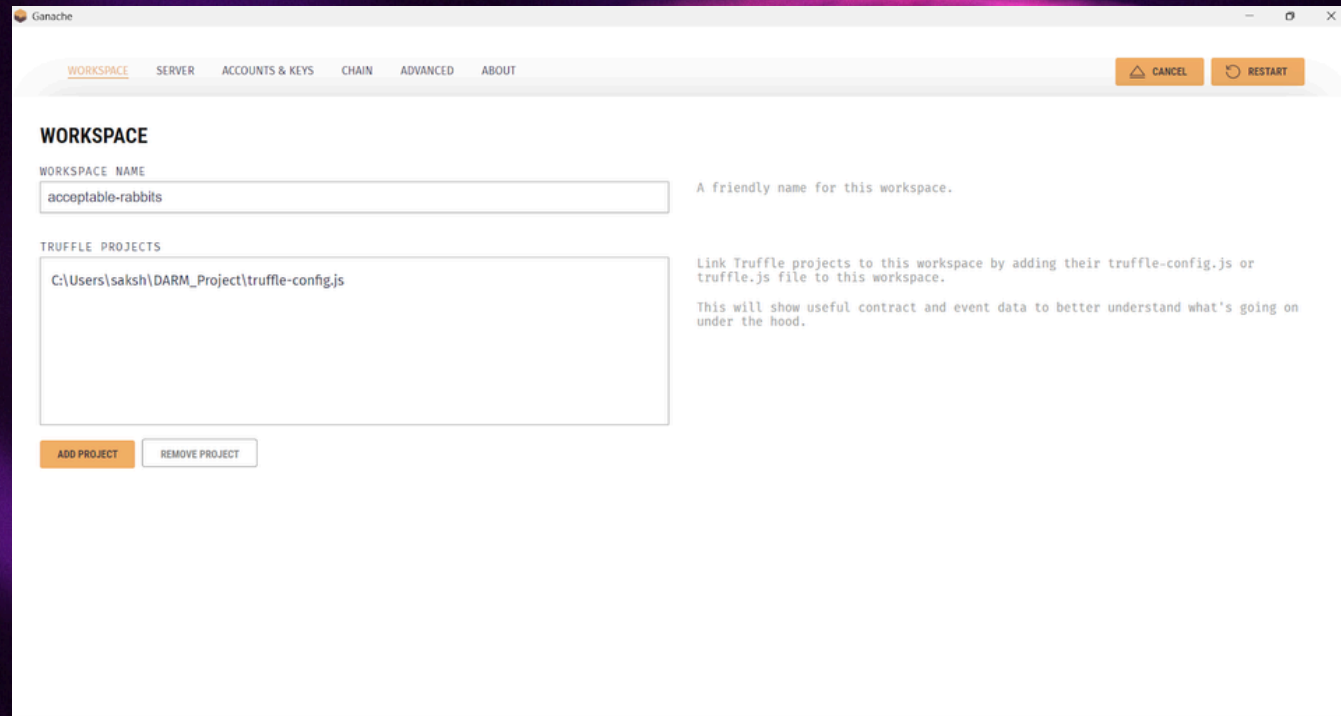
> Copying project files to C:\Users\saksh

Init successful, sweet!

Try our scaffold commands to get started:
  $ truffle create contract YourContractName # scaffold a contract
  $ truffle create test YourTestName        # scaffold a test

http://trufflesuite.com/docs
```

STEP 2: Start Ganache



Step 3: Compile Contracts

```

Compiling your contracts...
=====
> Compiling .\contracts\Campaign.sol
> Compiling .\contracts\ImpressionTracking.sol
> Compiling .\contracts\Migrations.sol
> Compiling .\contracts\PublisherRegistry.sol
> Compiling .\contracts\RewardDistribution.sol
> Artifacts written to C:\Users\saksh\DARM_Project\build\contracts
> Compiled successfully using:
  - solc: 0.8.21+commit.d9974bed.Emscripten.clang

> Duplicate contract names found for PublisherRegistry.
> This can cause errors and unknown behavior. Please rename one of your
contracts.

```

STEP 4: Deploy Contracts

```

Starting migrations...
=====
> Network name:      'development'
> Network id:        5777
> Block gas limit: 6721975 (0x6691b7)

1_initial_migration.js
=====

Replacing 'Migrations'
-----
> transaction hash:      0x8ce4f382c069842abeced25c1c354176a0a43cc78cd9e
c425bbfe4049cebdad12
> Blocks: 0              Seconds: 0
> contract address:      0x39F60c297774780d9B21f81E67fC7a692C5C5485
12
> block number:          12
> block timestamp:       1774691968
> account:               0xd77924a3dbfE614A9E8adE89cb0ce2f26E380F0B
> balance:               99.989029607083722192
> gas used:              241832 (0x3b0a8)
> gas price:             2.733812787 gwei
> value sent:            0 ETH
> total cost:            0.000661123413905784 ETH

> Saving migration to chain.
> Saving artifacts
-----
> Total cost:            0.000661123413905784 ETH
> Saving migration to chain.
> Saving artifacts

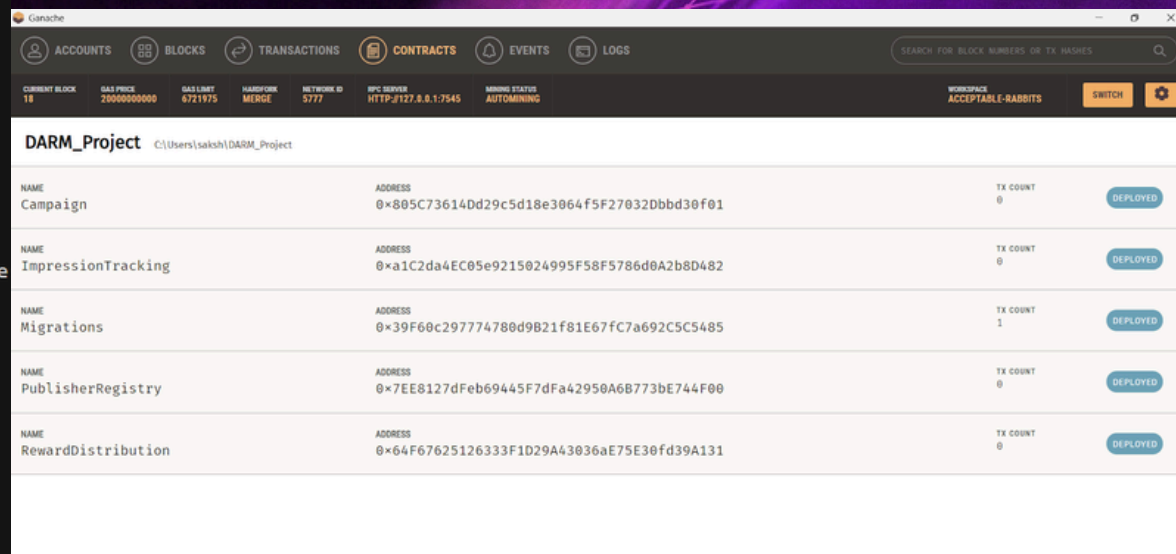
```

```

> Total cost:      0.007482338355330898 ETH

Summary
=====
> Total deployments: 5
> Final cost:      0.008143461769236682 ETH

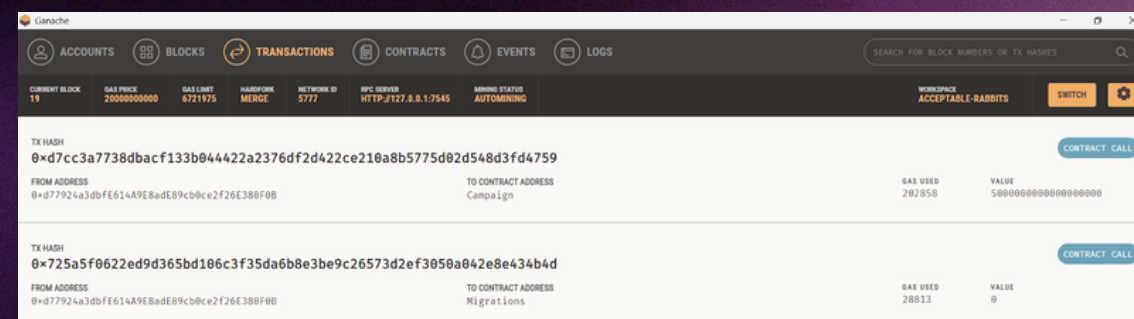
```



STEP 5: Open Truffle Console and interact with contracts

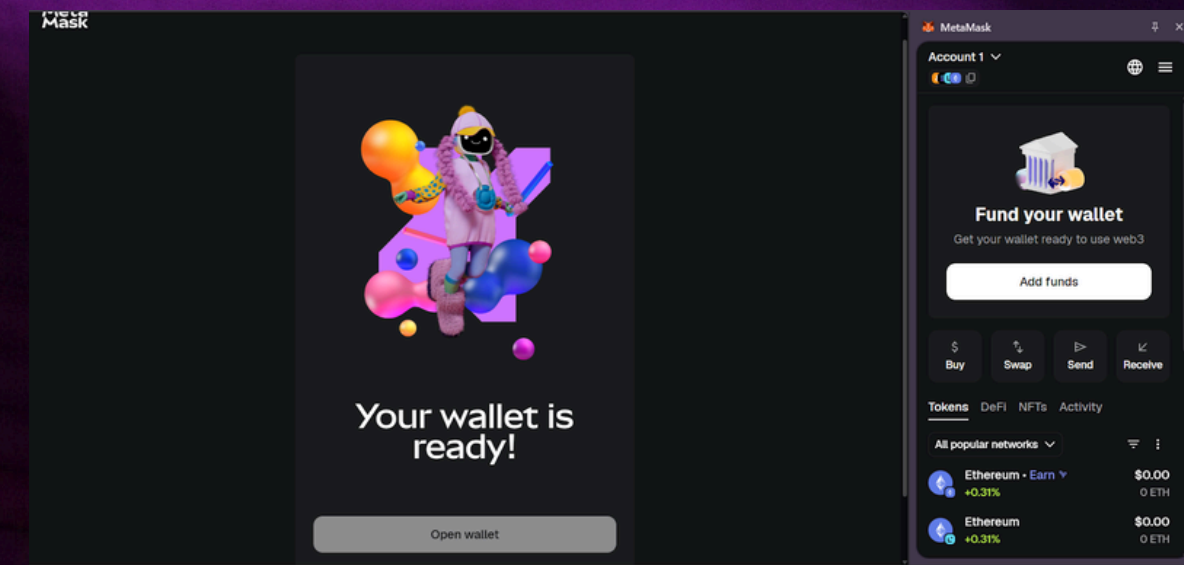
[illegible]

```
PS C:\Users\saksh\IOWARML_Protect> truffle console
truffle(development)> await publisher.registerPublisher('healthblog.com', { from: accounts[1] })
{
  tx: '0x1380fba25781d4bcala396ec075d3ef21e653fb095a3b755d1c8ad5408eb7b23',
  receipt: {
    transactionHash: '0x1380fba25781d4bcala396ec075d3ef21e653fb095a3b755d1c8ad5408eb7b23',
    transactionIndex: 0,
    blockNumber: 20,
    blockHash: '0xa97cf2726b20a9532ed8e9d5288ae50063522f383861055bc9a4fab8aadaa2b',
    from: '0x12dc8cb3b02ac4f4472ba829a3b46d18c94e1',
    to: '0x7Ee8127dfEb69445F7dfA42950a6B773be74F400',
    cumulativeGasUsed: 73795,
    gasUsed: 73795,
    contractAddress: null,
    logs: [ [Object] ],
    logIndex: 0
  },
  status: true,
  effectiveGasPrice: 2592349844,
  type: '0x2',
  rawLogs: [ [Object] ]
},
logs: [
  {
    address: '0x7EE8127dfEb69445F7dfA42950a6B773be74F400',
    blockHash: '0xa97cf2726b20a9532ed8e9d5288ae50063522f383861055bc9a4fab8aadaa2b',
    transactionIndex: 20,
    blockNumber: 20,
    logIndex: 0,
    removed: false
  }
]
```

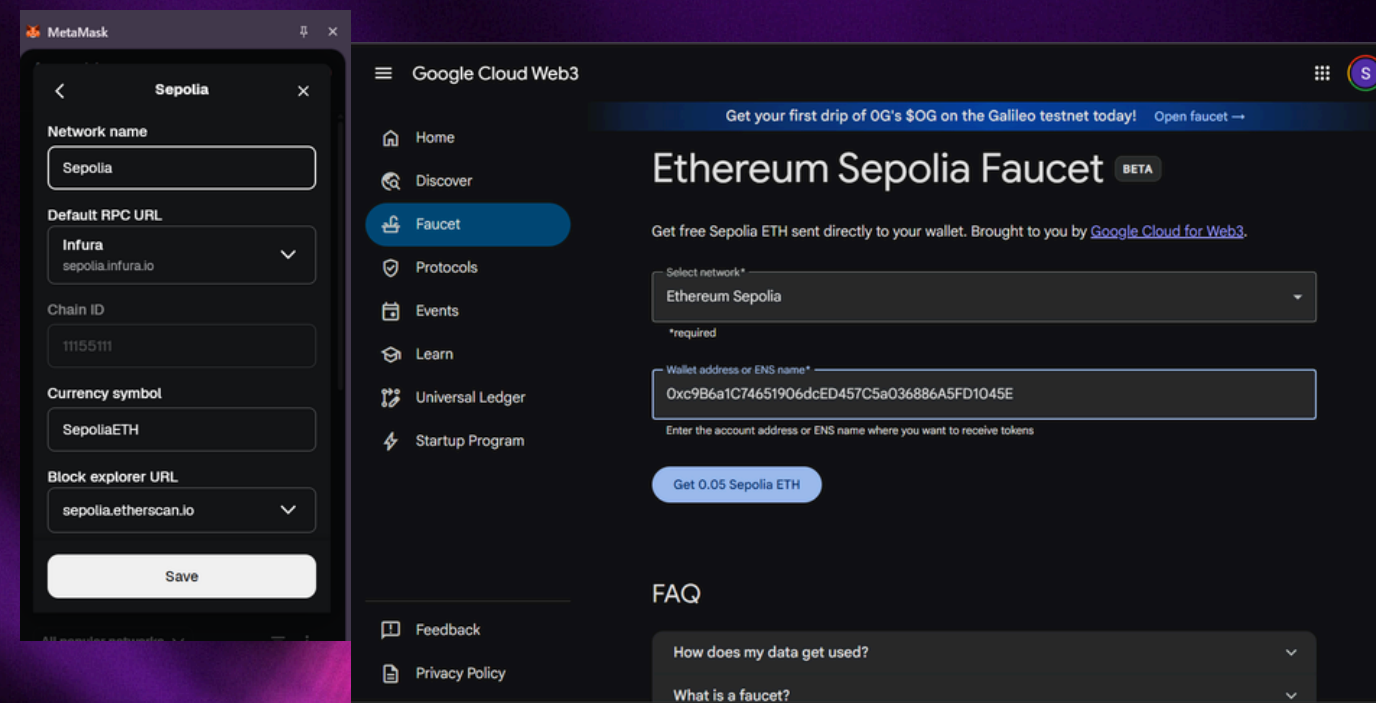


METAMASK WALLET INTEGRATION & LIVE TRANSACTIONS

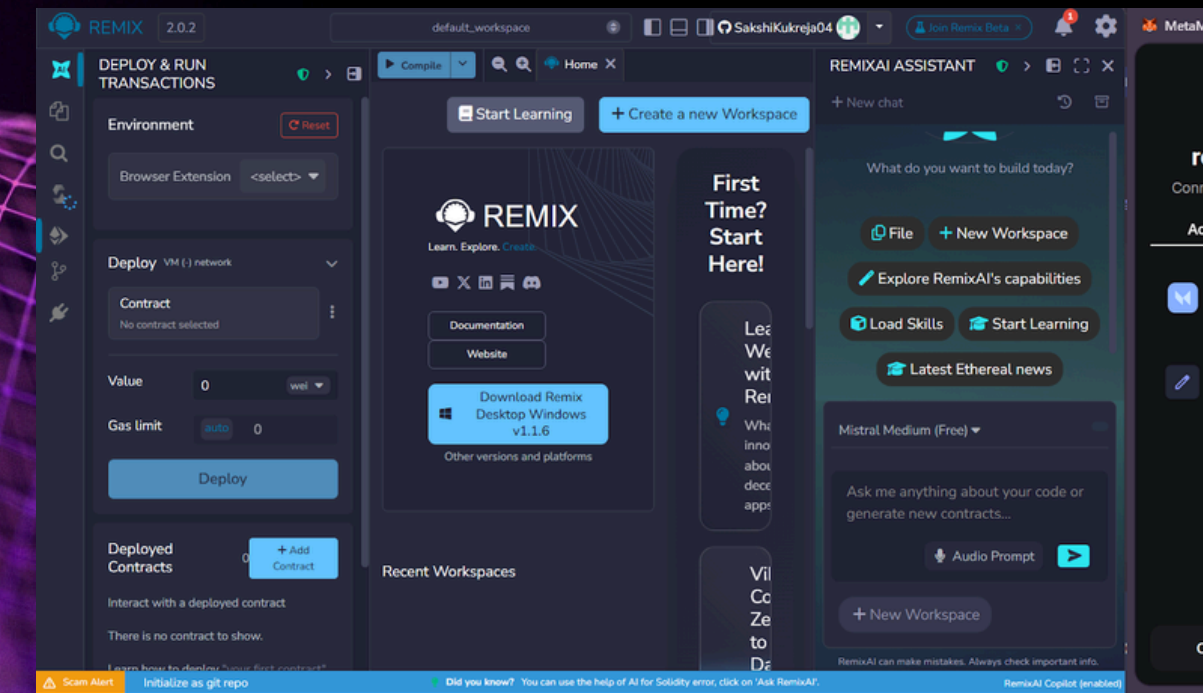
STEP 1 — SET UP METAMASK



STEP 2 — Wallet Funded



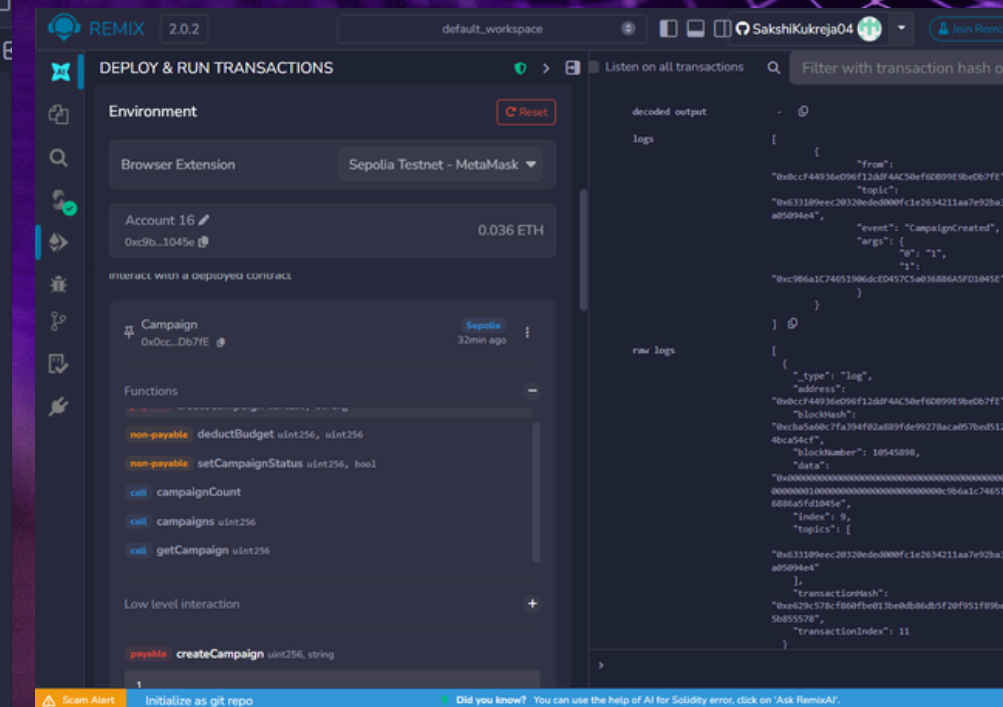
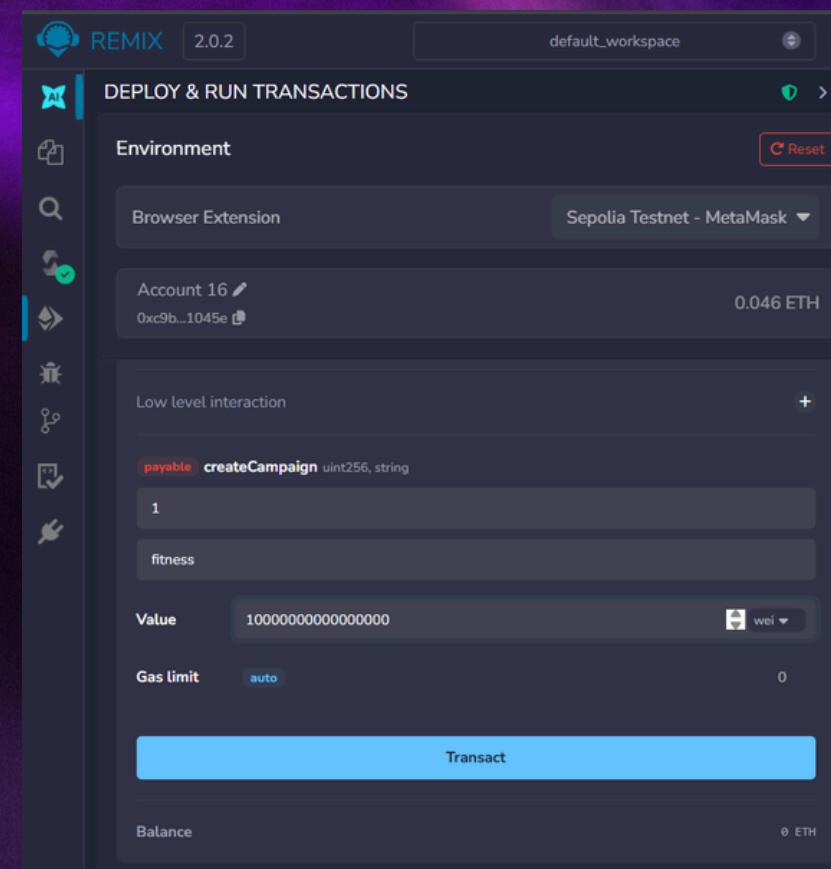
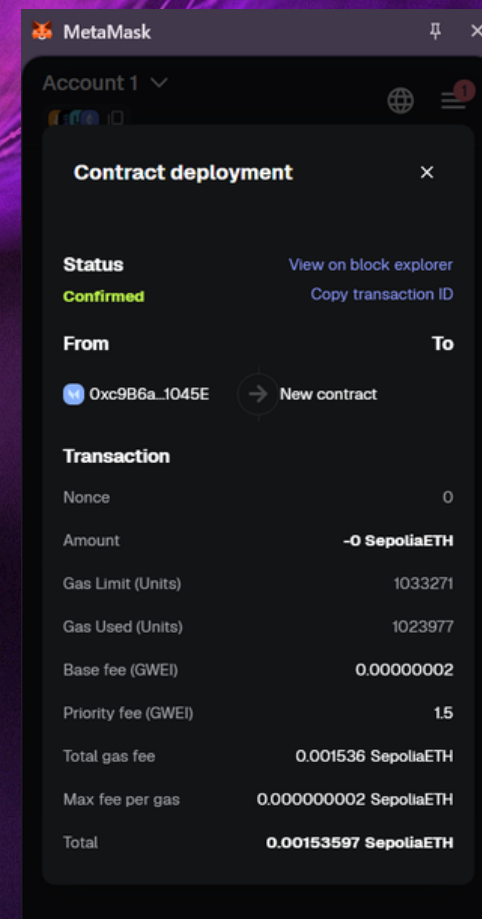
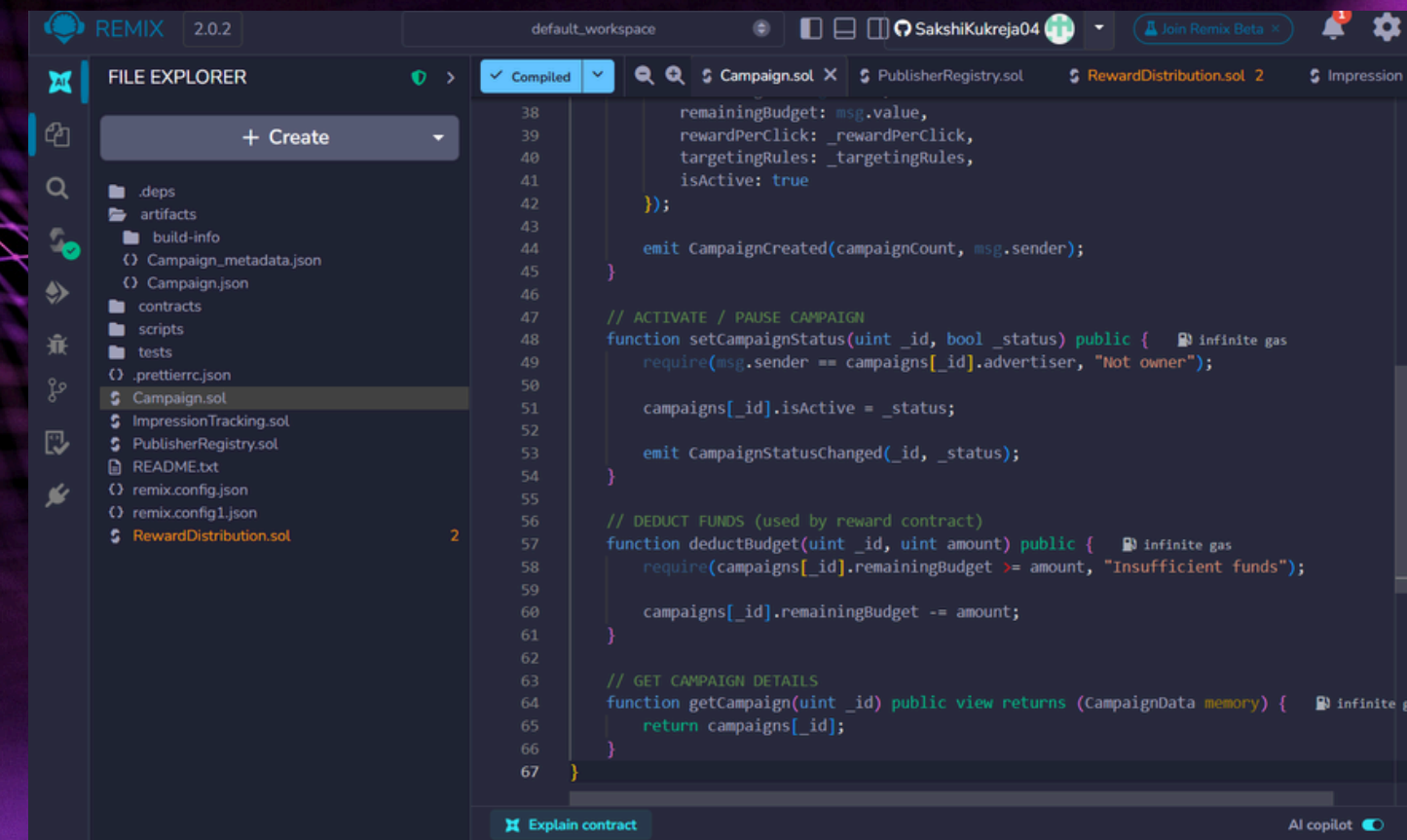
STEP 3 — Remix Connected



STEP 4 — Smart Contracts Created

STEP 6 — Contract Deployment

STEP 7: Interact with Contract



IMPLEMENTATION OF DAPP-IMPLEMENT THE BLOCKCHAIN PLATFORM GANACHE

1. Accounts created in Ganache

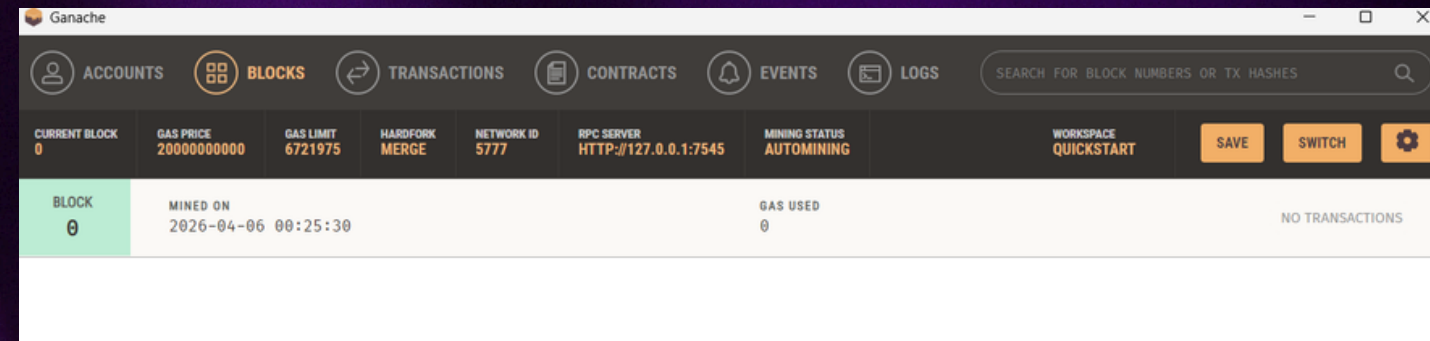
```
PS C:\Users\saksh\BARM_Project> ganache

ganache v7.9.2 (@ganache/cli: 0.10.2, @ganache/core: 0.10.2)
Starting RPC server

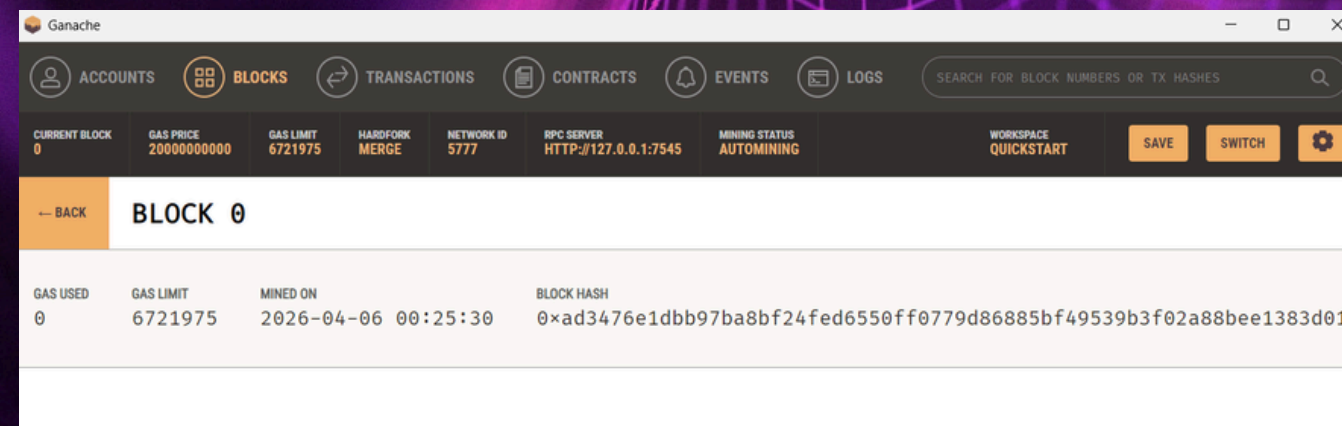
Available Accounts
=====
(0) 0xeCEa45C4e6a4eDbD4FEc0856B909a93c6EF96165 (1000 ETH)
(1) 0x430B4e78F1478ab04d67ddd75f30c4697aEA0da4 (1000 ETH)
(2) 0x4e288149FAB627f730C470D5214aE4fDEbDD7C10 (1000 ETH)
(3) 0xe4d9659420410AEa418D173C48edEb75DEB09184 (1000 ETH)
(4) 0x70afa82647E557AFB9760FE5719109BC7974FF52 (1000 ETH)
(5) 0xFd3B1F21C5BF2dEab5F69F6f39e50DDd002A2d32 (1000 ETH)
(6) 0xD48BD7adec67E27B646fa9A02aF0758C3D89579 (1000 ETH)
(7) 0xE61E7d64E03F663389Ffc73245D19C00f356E9ff (1000 ETH)
(8) 0x4b9877639ACA9ec0F14A21Cd1B9DF005AC9dBDf5 (1000 ETH)
(9) 0x06E2bB2fe569224d29712693e6d8adFaEca27202 (1000 ETH)

Private Keys
=====
(0) 0x6221718b1d0895886e3925aafb6b21e2a28b2b21d25899876452b5042d727c86
(1) 0x7e766c6e3e53b380655eba2cfff4d44079aa55c66a6641dccf5e190dba7009a52
(2) 0xb90e5a0ef52240637e013c791d443eee2de01e619b3c25ee52260cdd3ee9f676
(3) 0x0b64c921c883641a6d00298f4a74ee9b0a543fad98993f4983d636bf11f6eeaa6
(4) 0x2b6e216bc90a9df279143cfd2c4545cbabc09d62781b65c87c65bff82333095f
(5) 0xf06249723cb15c94328d8e8050c70b5c2ba6a0f209994cf908fd45c743635ed8
(6) 0xac97ef60d2f2b5edd9786ee1c735feabd8e1779d6b0d1abc3be7e177aeb9902a
(7) 0x5cc99105d79060489c50184db05a54ba438fdb793a211271bcad1566463edcae
(8) 0x8c8a43789936a253ed2bcc898a25df2e2cf835819aef974f04045f253fbff26c
(9) 0xf92eca3e2a405df53e4c03d765798aec9cabd818c82a6c7539bf733857a06f12
```

2. Genesis Block created



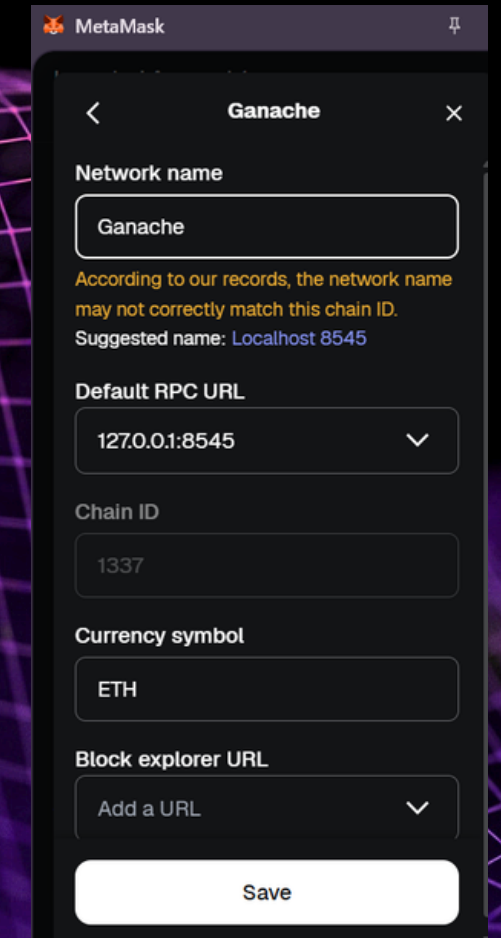
3. Contents of Block 0



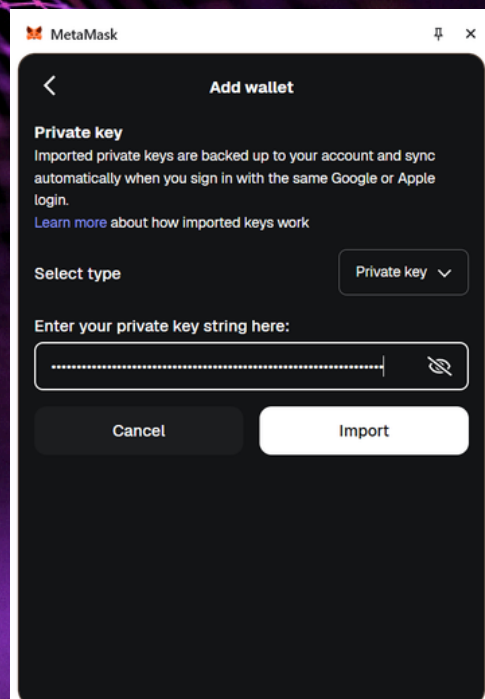
4. Log

```
RPC Listening on 127.0.0.1:8545
eth_chainId
eth_chainId
eth_getBlockByNumber
eth_chainId
eth_getBlockByNumber
eth_chainId
eth_accounts
eth_getBalance
eth_chainId
eth_getBalance
eth_chainId
eth_getBalance
eth_chainId
eth_getBalance
eth_chainId
eth_getBalance
eth_chainId
eth_getBalance
```

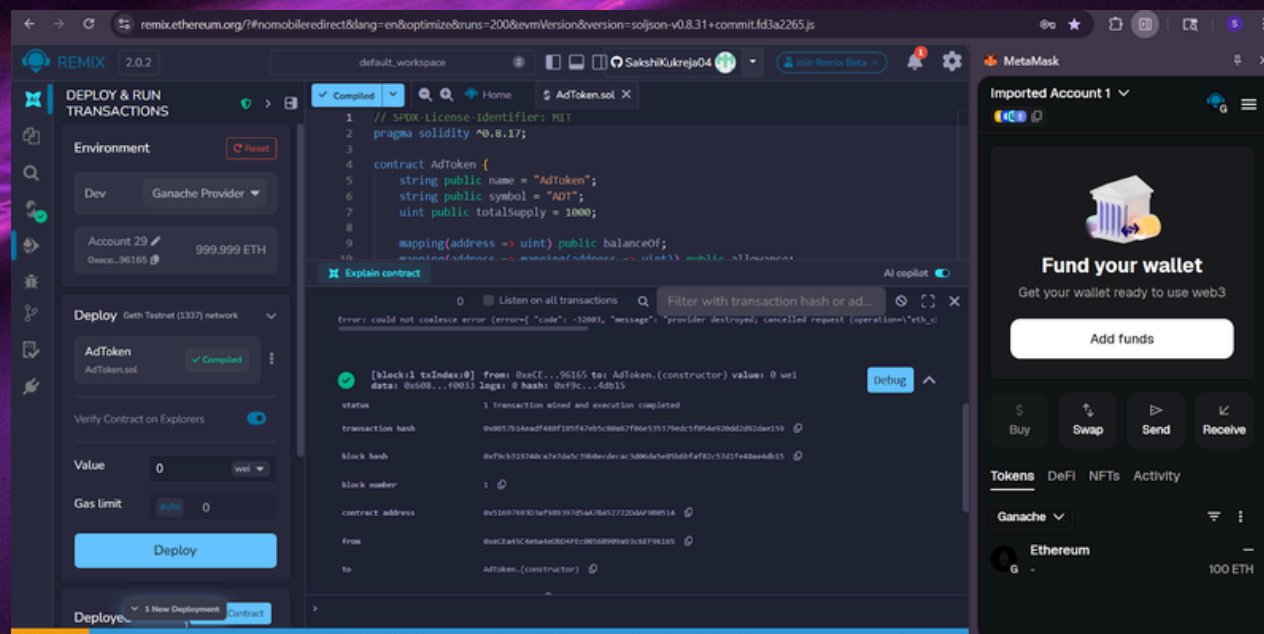
5. Adding Ganache Test Network



6. Import Account from Ganache—



7. Connect Remix IDE with MetaMask / Ganache

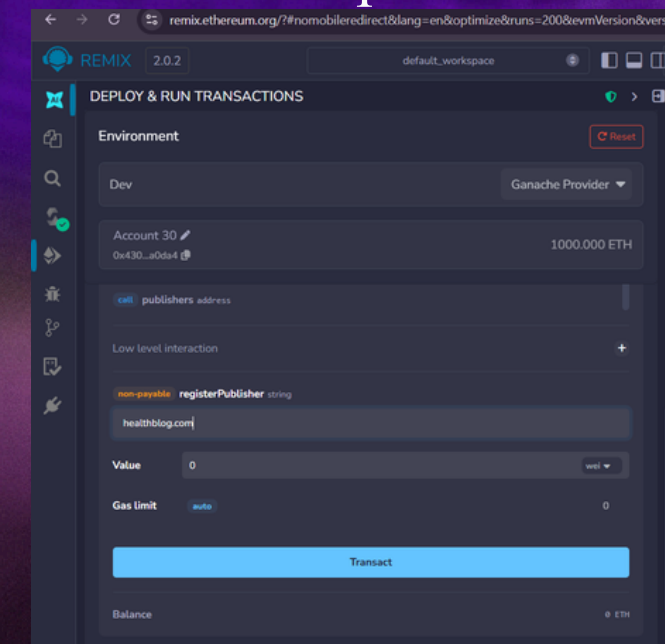


8. Transaction Count for First Account

Transaction: 0xb349fd8a95d695d8365a44fff362c2e928a501d3e9c7ee6a3fd93fdd061c6ce5
Contract created: 0xe75e2e10a3390afc69a341fb5a16a8152a16a123
Gas usage: 889734
Block number: 2
Block time: Mon Apr 06 2026 00:46:11 GMT+0530 (India Standard Time)

Transaction: 0xfbfced79610dcacb791a430e2e2321c5bfb72013a9c121ca751525b9bcb27609
Contract created: 0x55f91e2eb9af7b921f848611f31fcd8711005bd7
Gas usage: 326106
Block number: 3
Block time: Mon Apr 06 2026 00:47:00 GMT+0530 (India Standard Time)

9. Function 1: approve() switch to pub acc



REFERENCES

- [1] Blockchain and Smart Contracts for Transparent and Fraud-Resistant Digital Advertising(2025).
- [2] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger (Ethereum Yellow Paper),” 2014.
- [3] Blockchain Technology for Transparent Digital Advertising: Reducing Ad Fraud and Increasing Trust
- [4] F. Vogelsteller and V. Buterin, “ERC-20 Token Standard,” Ethereum Improvement Proposal (EIP-20), 2015. [Online]. Available: Fabian Vogelsteller and Vitalik Buterin, “ERC-20 Token Standard,” Ethereum Improvement Proposal (EIP-20), 2015.
- [5] THEMIS: A Decentralized Privacy-Preserving Ad Platform with Reporting Integrity
- [6] M. Swan, Blockchain: Blueprint for a New Economy. Sebastopol, CA, USA: O’Reilly Media, 2015.